

目录

阶段 04 实验报告	2
一、实验基本信息	2
(一) 实验目的	2
(二) 实验环境	2
二、实验核心原理	2
(一) 特权级切换机制	2
(二) 进程结构体定义	2
(三) 陷阱处理流程	3
三、实验步骤与关键代码	3
(一) 任务一：proczero 的定义和初始化	3
(二) 任务二：用户态陷阱处理	4
四、实验关键问题解答	5
(一) 进程抽象相关问题	5
(二) ZOMBIE 状态的作用	5
(三) 进程表大小限制的影响	5
(四) 防止进程 ID 重复的策略	6
(五) fork() 相关问题	6
(六) 用户态陷阱处理流程拆分原因	6
五、实验结果与总结	6
(一) 实验结果	6
(二) 实验总结	6

阶段 04 实验报告

[仓库地址在此处](#)

此阶段代码请通过 `git checkout userinit` 来查看。

一、实验基本信息

（一）实验目的

1. 深入理解 xv6 操作系统的进程管理机制，掌握进程的创建、初始化及特权级切换原理。
2. 实现首个用户态进程 `proczero` 的创建与初始化，完成用户态与内核态的陷阱处理流程。
3. 解答实验指导中提出的核心问题，深化对进程抽象、生命周期及系统调用的理解。

（二）实验环境

- 操作系统：xv6
- 架构：RISC-V
- 开发工具：GCC、QEMU（模拟器）、汇编器、链接器

二、实验核心原理

（一）特权级切换机制

1. U 态到 S 态切换：触发 `trap` 时（除时钟中断），硬件自动禁用中断、保存 `pc` 到 `sepc`、设置特权级为 `supervisor`，跳转至中断服务程序入口 `stvec`。内核需手动切换页表和栈，保存其他寄存器。
2. S 态到 U 态切换：手动清除 `sstatus` 的 `SPP` 字段、启用用户中断（`SPIE=1`）、设置 `sepc` 为用户进程 `PC`，恢复用户页表后执行 `sret` 指令，硬件自动恢复 `pc` 和用户态状态。

（二）进程结构体定义

xv6 中进程结构体核心字段及作用如下：

```
1 typedef struct proc {
2     int pid;                // 进程唯一标识符
3     pgtbl_t pgtbl;          // 用户态页表，映射虚拟地址到物理地址
4     uint64 heap_top;        // 用户堆顶地址，用于动态内存扩展
5     uint64 ustack_pages;    // 用户栈占用页面数量
6     trapframe_t* tf;        // 特权级切换时的运行环境暂存区，保存寄存器信息
7     uint64 kstack;          // 内核栈虚拟地址，进程在内核态运行时使用
8     context_t ctx;          // 内核态进程上下文，用于进程切换
9 } proc_t;
```

（三）陷阱处理流程

用户态陷阱处理分为四个阶段：`user_vector` → `trap_user_handler` → `trap_user_return` → `user_return`，拆分设计是为了兼容进程从内核态初始化后进入用户态的路径。

三、实验步骤与关键代码

（一）任务一：proczero 的定义和初始化

1. 页表初始化（proc_pgtbl_init）

功能：创建用户态页表，完成 trampoline、内核栈等关键区域的映射。

```

1  pgtbl_t proc_pgtbl_init(uint64 trapframe) {
2      // 分配页表物理内存
3      pgtbl_t pgtbl = (pgtbl_t)pmem_alloc(PGSIZE);
4      memset(pgtbl, 0, PGSIZE); // 全局数据区初始化为0，避免异常
5
6      // 映射trampoline页（内核与用户共享，用于上下文切换）
7      map_page(pgtbl, TRAMPOLINE, (uint64)trampoline, PTE_R | PTE_X);
8
9      // 映射内核栈（每个进程独立内核栈）
10     uint64 kstack = KSTACK((uint64)myproc());
11     map_page(pgtbl, kstack, (uint64)pmem_alloc(PGSIZE), PTE_R | PTE_W);
12
13     // 映射用户代码段、数据段及陷阱帧
14     map_page(pgtbl, USER_CODE, (uint64)initcode, PTE_R | PTE_X | PTE_U);
15     map_page(pgtbl, (uint64)trapframe, (uint64)trapframe, PTE_R | PTE_W |
16         PTE_U);
17     return pgtbl;
18 }
```

2. 首个进程创建（proc_make_first）

功能：初始化 proczero 的字段，切换上下文将 CPU 使用权交给该进程。

```

1  void proc_make_first() {
2      proc_t *p = myproc();
3      p->pid = 0; // 首个进程PID设为0
4
5      // 1. 初始化用户态页表
6      p->tf = (trapframe_t*)pmem_alloc(PGSIZE);
7      memset(p->tf, 0, PGSIZE);
8      p->pgtbl = proc_pgtbl_init((uint64)p->tf);
9
10     // 2. 初始化用户栈（栈指针指向栈顶，向下生长）
11     p->ustack_pages = 1;
12     uint64 ustack = USER_STACK - PGSIZE;
```

```

13     map_page(p->pgtbl, ustack, (uint64)pmem_alloc(PGSIZE), PTE_R | PTE_W
    | PTE_U);
14     p->tf->sp = USER_STACK; // 栈指针初始化为栈顶地址
15
16     // 3. 设置陷阱帧和上下文, 指定进程入口地址
17     p->tf->epc = USER_CODE; // 进程入口为用户代码起始地址
18     p->ctx.ra = (uint64)trap_user_return; // 上下文切换返回地址
19     p->ctx.sp = p->kstack + PGSIZE; // 内核栈指针指向栈顶
20
21     // 4. 切换上下文, 将CPU控制权交给proczero
22     cpu_t *c = mycpu();
23     c->proc = p;
24     swtch(&c->ctx, &p->ctx);
25 }

```

(二) 任务二：用户态陷阱处理

1. 陷阱处理函数 (trap_user_handler)

功能：响应用户态系统调用，打印调用信息。

```

1 void trap_user_handler() {
2     proc_t *p = myproc();
3     uint64 scause = r_scause();
4
5     // 处理系统调用 (scause=8表示系统调用)
6     if (scause == 8) {
7         printf("get a syscall from proc %d\n", p->pid);
8         // 设置系统调用返回值, 清除系统调用编号
9         p->tf->a0 = 0;
10        p->tf->a7 = 0;
11    }
12
13    // 其他陷阱类型处理 (如异常)
14    else {
15        panic("unknown user trap: scause=%p", scause);
16    }
17 }

```

2. 陷阱返回函数 (trap_user_return)

功能：恢复用户态寄存器信息，执行 sret 指令返回用户态。

```

1 void trap_user_return() {
2     proc_t *p = myproc();
3     cpu_t *c = mycpu();
4
5     // 恢复用户页表
6     w_satp(MAKE_SATP(p->pgtbl));
7     sfence_vma(); // 刷新页表缓存

```

```
8
9    // 恢复用户态寄存器（从trapframe读取）
10   restore_trapframe(p->tf);
11
12   // 设置sstatus寄存器：启用用户中断，清除SPP字段
13   w_sstatus((r_sstatus() & ~SSTATUS_SPP) | SSTATUS_SPIE);
14   // 设置返回地址为用户进程断点
15   w_sepc(p->tf->epc);
16
17   // 执行sret返回用户态
18   asm volatile("sret");
19 }
```

四、实验关键问题解答

（一）进程抽象相关问题

1. 进程结构体字段作用：

- `lock`：保证进程操作的原子性，防止并发访问冲突。
- `state`：标识进程当前状态（UNUSED/RUNNABLE/RUNNING 等），用于调度器管理。
- `chan`：等待通道，用于进程睡眠时挂起等待资源。
- `trapframe`：保存特权级切换时的寄存器信息，实现内核态与用户态上下文切换。
- `context`：保存进程切换时的内核态上下文，用于调度器切换进程。

2. 进程状态转换触发条件：

- UNUSED→USED： `allocproc()` 分配进程结构时。
- USED→RUNNABLE：进程初始化完成（如 `fork()` 结束）或被唤醒（如等待资源可用）。
- RUNNABLE→RUNNING：调度器选中进程，执行 `swtch()` 切换上下文。
- RUNNING→SLEEPING：进程调用 `sleep()` 等待资源（如等待 IO 完成）。
- RUNNING→ZOMBIE：进程调用 `exit()` 终止，等待父进程回收资源。

3. 锁保护的原因：多 CPU 环境下，进程操作（如状态修改、资源访问）可能被并发执行，锁可避免数据竞争和不一致。

（二）ZOMBIE 状态的作用

ZOMBIE 状态用于保存进程终止后的退出状态（`xstate`），等待父进程通过 `wait()` 函数获取该状态并回收进程资源。若没有该状态，父进程无法获取子进程终止信息，会导致资源泄露（如进程表项、内存空间无法释放）。

（三）进程表大小限制的影响

进程表大小限制了系统同时运行的最大进程数。若限制过小，多任务场景下会出现进程无法创建的情况；若过大，会占用过多内核内存（进程表存储在内存空间），降低系统资源利用率。

（四）防止进程 ID 重复的策略

通过维护一个全局 PID 计数器，每次分配 PID 时自增，同时检查当前 PID 是否已被占用（遍历进程表）。当 PID 达到最大值时，循环从头开始查找未使用的 PID，确保分配的 PID 唯一。

（五）fork() 相关问题

1. 父子进程返回值不同的原因：fork()通过复制父进程创建子进程，子进程创建成功后，内核会修改子进程的返回值为 0，父进程返回值为子进程 PID，便于父子进程区分执行路径。
2. 内存复制实现方式：fork()通过复制父进程的页表和物理内存实现内存共享，xv6 中采用写时复制（Copy-On-Write）优化，初始时父子进程共享物理内存，仅当某一方修改内存时才复制对应页面，减少内存开销。
3. fork()性能瓶颈：内存复制过程占用大量 CPU 和内存资源，尤其是大内存进程复制时耗时显著。优化方案是采用写时复制，延迟内存复制时机，仅在必要时执行。

（六）用户态陷阱处理流程拆分原因

用户进程初始创建于内核态（proc_make_first()在 kernel 中执行），需通过 trap_user_return→user_return 直接进入用户态，无需经过 user_vector 入口。拆分流程可兼容“内核态初始化→用户态”和“用户态陷阱→内核态处理→用户态”两条路径，确保进程正常切换。

五、实验结果与总结

（一）实验结果

成功创建首个用户态进程 proczero，该进程执行两次 sys_print 系统调用，系统打印对应调用信息后，进程进入死循环。0 号 CPU 陷入用户态死循环，其余 CPU 陷入 main() 末尾死循环，符合实验预期。

（二）实验总结

本次实验通过实现首个用户态进程的创建与陷阱处理，深入理解了 xv6 的进程管理和特权级切换机制。关键收获包括：掌握进程结构体初始化、页表映射、上下文切换的核心流程；理解用户态与内核态陷阱处理的差异及设计逻辑；解答了进程状态转换、ZOMBIE 状态作用等核心问题，为后续多进程调度和系统调用优化奠定基础。